

① 3 REGEX that match any non-negative nr. multiple of 5.

- 1) $^{\wedge}[0-9]^*[05]^\$$
 - $^{\wedge}$: start of string
 - $[0-9]^*$: any digit, repeat previous part 0 or more times
 - $[05]$: final digit 0 or 5
 - $^\$$: end of string
- 2) $^{\wedge}(0|[1-9][0-9]^*[05])^\$$
 - $^{\wedge}$: start string
 - $(0|[1-9][0-9]^*[05])$: group
 - 0 : accept 0 on its own
 - $[1-9]$: first digit $\neq 0$
 - $[0-9]^*$: repeat previous part 0 or more times (only the $[0-9]$)
 - $[05]$: final digit 0 or 5
 - $^\$$: end of string
- 3) $^{\wedge}[[[:digit:]]^*[05]^\$$
 - $^{\wedge}$: start string
 - $[[[:digit:]]^*$: posix digit class, 0 or more digits
 - $[05]$: final digit 0 or 5
 - $^\$$: end of string

② 5 GREP commands that display all lines in a file that contain "a" or "A".

- 1) `grep -i 'a' filename`
 - `grep`: search lines
 - `-i`: ignore case
 - `'a'`: search this letter
 - `filename`: input file
- 2) `grep -Ei "a" filename`
 - `grep`: search lines
 - `-E`: extended regex
 - `-i`: ignore case
 - `"a"`: search this letter
 - `filename`: input file

(not really needed extended regex)
- 3) `grep "[aA]" filename`
 - `grep`: search lines
 - `"[aA]"`: regex pattern, character class (match one character from inside) (either a or A)
 - `filename`: input file
- 4) `grep -E "[aA]^+" filename`
 - `grep`: search lines
 - `-E`: extended regex (needed because of `^+`)
 - `"[aA]^+"`: regex pattern, character class, one or more occurrences
 - `filename`: input file
- 5) `grep -e "a" -e "A" filename`
 - `grep`: search lines
 - `-e "a"`: first pattern (lowercase a)
 - `-e "A"`: second pattern (uppercase A)
 - `filename`: input file

③ 2 SED commands that display from a file only the lines not containing

1) `sed -E "/7/d" filename`
 Stream editor command (not needed) → extended regex
 search lines having digit 7 → delete those lines

2) `sed -m "/7/!p" filename`
 don't print lines automatically → select lines containing digit 7 → negate selection ⇒ print ⇒ lines NOT containing 7.

④ 1 AWK command that displays the sum of the next to last (penultimate) field of all lines.

`awk '{SUM += $(NF-1)} END {print SUM}' filename`
 "not" → field operator → print final sum.
 field-processing command → variable → add to current value → \$(penultimate field index) → value of penultimate field index
 " → protect \$ from shell.
 runs after all lines are processed

⑤ How to redirect on the cmd line the standard error through pipe to another program?

cmd line → standard error → pipe → another program → cmd line.
 command 2>&1 | another-program
 first cmd. → stderr file descriptor → redirect → same place as stdout → receives the output
 } send both stdout and stderr through the pipe.
 command 2>&1 > /dev/null | another-program
 ↓ discard normal stdout
 } send only stderr through the pipe.

⑥ Shell script displaying all cmd-line arguments without using FOR.

`#!/bin/bash` → shebang / interpreter
`echo "$@"` → print text → all command-line args.
 | `#!/bin/bash`
 | `echo`
 | `printf "%s\n" "$@"`
 | one arg / line.

Draw process hierarchy (including parent)

```
for (i=0; i<3; i++) {
```

```
    fork();
```

```
    execlp("ls", "ls", "/", NULL);
```

```
}
```

list of arguments
got finished.

P
|
C1

file to
find

first
arg. passed
to the program
is its own name

root directory

⑧ Add code s.t. file "b.txt" is overwritten with file "a.txt"

`execlp("cat", "cat", "a.txt", NULL)` → given instruction.

→ redirect cat a.txt into b.txt:
target output file

if missing,
create file.

erase old
content

OWNER → R/W
GROUP → R
permissions
OTHERS → R.

```
int fd = open("b.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```
dup2(fd, STDOUT_FILENO);
```

 → duplicate fd over stdout

```
close(fd);
```

 → close extra descriptor
↳ standard output, descriptor 1.

```
execlp("cat", "cat", "a.txt", NULL);
```

execute program

to find

program's
name

input file that
cat will read
(argument)

⑨ Why not bidirectional FIFO?

We need to create two fifos for bidirectional communication because FIFO is a NAMED PIPE and one FIFO means one byte stream, therefore A → B one direction, B → A another direction. Without 2 fifos, deadlocks and race conditions might appear.

Correct pattern: f1: A writes, B reads

f2: B writes, A reads.

⑩ How many FIFOs can one process open alone?

→ Zero FIFOs. because { open FIFO for reading waits for writer
open FIFO for writing waits for reader.

• if no other process ever opens the other end, the first blocking open waits forever.

→ as many as files descriptor limit → only for special cases like non-blocking / open read-write

`ulimit -m` command

⑪ When would you prefer using a process instead of a thread?

Use processes when:

- need isolation (separate memory)

- security

- separate program via exec

- less risk of shared-memory corruption

→ Processes have their own memory space, reducing memory corruptions

→ Processes can be distributed to multiple machines (threads are limited to a single machine's resources)

→ Processes offer better security boundaries, since they don't share the same memory (=the same address space)

→ A crash in one process does not affect the outcome of the other.

⑫ What is a "critical section"?

Critical section = code that accesses/modifies shared resources.

ex:

```
pthread_mutex_lock(&m);  
m++;  
pthread_mutex_unlock(&m);
```

= section in the program where shared resources (such as memory variables) are accessed and potentially modified. ⇒ we have to prevent its instability through mutual exclusion (mutexes) to ensure consistent data behaviour.

⑬ Why should the thread recheck the condition after returning from the `pthread_cond_wait` call?

Situation:

```
pthread_mutex_lock(&m);  
← while (condition == false) {  
    pthread_cond_wait(&cond, &m);  
}  
pthread_mutex_unlock(&m);
```

Recheck because:

- signal does not guarantee condition is true
- another thread may ^{have} changed the state already.

A thread needs to recheck to be sure the thread can enter correctly and the condition is still true because of signals from the other threads that could have occurred.

Effect of replacing calls to pthread_mutex_lock with calls to pthread_rwlock_rdlock

pthread_mutex_lock → one thread enters

pthread_rwlock_rdlock → many readers enter

If critical section modifies data, rdlock is wrong because many threads can write at the same time ⇒ no mutual exclusion
⇒ data races. ↗ why param?

⇒ correct way would be: pthread_rwlock_wrlock(&rwlock);

write lock is exclusive.

(15) Effect of calling pthread_barrier_wait on a barrier initialised with 1?

⇒ immediately release the waiting thread because the barrier condition is already satisfied. No waiting, so thread will proceed.

Barrier → waits for new arrivals

barrier count = 1 ⇒ needs one arrival before proceeding.

barrier count = 2 ⇒ needs two arrivals before...

✱

⋮

Mutex → protects a critical section.

(16) How to increment the value of a POSIX semaphore?

✓ sem_post(&sem) → increases semaphore counter by 1.

↓
semaphore variable

sem_wait(&sem) → decrease sem. counter by 1
() wait if counter is 0.

(17) How to prevent deadlocks?

- consistent locking order
- avoid nested locks
- resource hierarchies

- use trylock
- release locks in reverse order
- short critical sections

Always acquire mutexes in the same global order.

(ex: thread 1 locks A then B)
thread 2 locks A then B)

(18) What state transition will a process undergo when reading from a file?

=> When a process reads from a file, it transitions from the running state to the blocked state if the data is not immediately available, then back to ^(waiting) ready state and running state when data is available.

running : process uses CPU

blocked : waits for I/O

ready : I/O done, waits for CPU

running : scheduled again

(19) Content of the superblock on a Linux disk?

SUPERBLOCK = stores filesystem metadata

↳ = "block 1"

↳ contains info:

- nr. of inodes
- nr. of zones
- inode allocation bitmap
- free-space bitmap
- zone size
- filesystem state

(20) Can you create a hard link towards a file on a $\neq$ partition?

Hard links cannot cross partitions because they reference their inodes, and inodes are unique to their filesystem.

! {
• hard link $\rightarrow$ inode
• inode $\in$ one filesystem
• $\neq$ partition $\Rightarrow \neq$ filesystem

=> Symbolic link can be used instead
(path)
 $\neq$
inode